astro-geo-backend — Technical Notes (Current Stage)

Alessandro Veltri

December 28, 2025

1 Scope and non-scope

This repository is a small backend whose job is to convert:

1. a **place query** (e.g. a city name),
2. a **local civil date and time**,

into a single deterministic JSON payload containing:

- WGS84 coordinates and an IANA timezone identifier,
- the corresponding UTC instant (with explicit DST ambiguity handling),
- minimal astronomical context: **IAU constellation at the zenith** and **Sun position/constellation**.

Non-scope: this project explicitly does *not* implement any astrological interpretation, horoscope logic, house systems, or narrative mapping. It only produces clean, reproducible inputs for downstream modules.

2 High-level pipeline

The current backend is a three-stage pipeline:

$$\texttt{GEO} \rightarrow \texttt{TIME} \rightarrow \texttt{ASTRO}$$

or in words:

a) GEO: resolve a place query to (ϕ, λ) and a tzid,

b) TIME: resolve (local date, local time, tzid) to a *unique* UTC instant,

c) ASTRO: compute zenith constellation and Sun context at that UTC instant.

The orchestration is performed by a single CLI entry point, which prints **exactly one JSON object** to stdout.

3 Repository structure (current)

The relevant components are organized under `src/`:

```
src/
|-- geo/         (place -> coords -> tzid)
|-- civil_time/  (local -> UTC, DST-safe)
|-- astro/       (zenith constellation, Sun position/constellation)
`-- main.py      (GEO -> TIME -> ASTRO orchestration, JSON output)
```

4 Stage 1: GEO — place query to coordinates (+ tzid)

4.1 Data source and constraints

The geocoding stage uses OpenStreetMap Nominatim via HTTPS GET queries. Because Nominatim is a shared public service, the module implements:

- **caching** of resolved queries to a JSON file in `$HOME/.cache/`,
- **rate limiting** (minimum delay between remote requests),
- a **user-agent string** (required by Nominatim usage policy).

4.2 Optional timezone resolution

Timezone resolution from coordinates is performed only if the optional Python dependency `timezonefinder` is installed. If it is not available:

- the GEO stage returns coordinates but **tzid is missing**,
- the main pipeline stops early with a clear error telling the user to install `timezonefinder`.

4.3 Cache behavior

A cache hit skips the network entirely. Additionally, old cache entries can be *upgraded*: if an entry has no timezone stored and `timezonefinder` is now available, tzid is computed and stored.

4.4 Inputs and outputs

Input: place query string.
Output: best match (currently `limit=1`) with:

$$\{\texttt{display\_name}, \texttt{lat}, \texttt{lon}, \texttt{tzid}\}.$$

5 Stage 2: TIME — local civil time to unique UTC instant

This stage solves a subtle problem: local civil times are not always one-to-one with UTC.

5.1 DST fall-back ambiguity (two valid instants)

During the autumn DST transition, a local wall-time can happen twice. The time module detects this by attempting both PEP 495 folds and verifying round-trips.

Policy options:

- `ambiguous='raise'`: error out (strict mode),
- `ambiguous='earliest'`: choose the earlier UTC instant,
- `ambiguous='latest'`: choose the later UTC instant.

5.2 DST spring-forward gaps (non-existent local times)

During the spring DST transition, a local wall-time can be invalid. Policy options:

- `nonexistent='raise'`: error out (strict mode),
- `nonexistent='shift_forward'`: move forward in small steps until valid,
- `nonexistent='shift_backward'`: move backward in small steps until valid.

5.3 Why this matters

Astronomical computations must be fed an **unambiguous instant**. This module ensures the pipeline always reaches a unique UTC time, while also allowing the CLI to:

- start strict (raise),
- fall back to a chosen policy,
- emit a **warning string** into the JSON payload when a policy is applied.

6 Stage 3: ASTRO — minimal astronomical context

All astronomical computations are performed with `astropy`.

6.1 Zenith constellation

Given:

$$\texttt{utc\_iso},\ \phi = \texttt{lat\_deg},\ \lambda = \texttt{lon\_deg},$$

the code constructs an `AltAz` frame at that Earth location and time, defines the zenith as ($\texttt{alt} = 90°,\ \texttt{az} = 0°$) (azimuth is arbitrary at zenith but fixed for determinism), transforms to ICRS, and queries the official IAU constellation boundaries.

Output can be either:

- full constellation name, or
- 3-letter IAU abbreviation (if `-short-constellation` is passed).

6.2 Sun position and constellation

At the same UTC instant, the Sun is obtained via `astropy.coordinates.get_sun`. The code then returns:

- Sun constellation (IAU boundaries),
- true ecliptic of date longitude/latitude,
- ICRS right ascension and declination.

7 The main CLI: single JSON payload

7.1 Command-line interface

The primary entry point is:

```
python3 src/main.py --city "Cosenza, Italy" --date 1982-08-25 --time 12:00
```

7.2 What it prints

The program prints **one JSON object** with three top-level blocks:

- `place`: `display_name`, `lat`, `lon`, `tzid`
- `time`: `local`, `utc`, `warnings`
- `astro`: zenith constellation + Sun block

This contract (one JSON object to stdout) is intentional: it allows the CLI to be used in pipes, scripts, and future services.

8 Determinism and reproducibility

8.1 What is deterministic

- Given **fixed** `(lat,lon,tzid,local time)`, the TIME stage is deterministic.
- Given **fixed** `(lat,lon,utc)`, the ASTRO stage is deterministic.
- Given **cached** geocoding entries, GEO is deterministic (no network).

8.2 What is not fully deterministic

- The GEO stage depends on an external service (Nominatim) *unless cached*.
- Place-name ambiguity is resolved by "best result" with `limit=1`, which can change if Nominatim ranking changes or the query is underspecified.

9 Strengths and flaws (current stage)

9.1 Strengths

- **Clean separation of concerns:** GEO, TIME, ASTRO are independent and testable.
- **DST-safe time resolution:** ambiguity and gaps are detected explicitly, with policies.
- **Single structured output:** one JSON object is easy to integrate downstream.
- **Graceful optional dependency:** timezone resolution is modular (`timezonefinder`).
- **IAU correctness:** constellation lookups use official IAU boundaries (via Astropy).

9.2 Flaws / known limitations

- **Geocoding is external:** without cache, GEO depends on network + Nominatim availability.
- **Timezone resolution is approximate:** `timezonefinder` uses polygons; edge cases near borders or in complex administrative regions can fail or be wrong.
- **Underspecified place queries:** "Parma" vs "Parma, Ohio" shows that tzid can be missing or ambiguous depending on the match quality.
- **Single-result geocoding:** `limit=1` hides alternatives; this is fine for a backend but not ideal for interactive UX.
- **No uncertainty metadata:** the JSON does not currently carry a confidence score beyond Nominatim "importance" (and even that is not surfaced to the user by default).
- **Astro scope is minimal:** zenith constellation and Sun context only; no Moon, planets, or local sky visibility constraints.

10 Practical next steps (suggested)

If the goal is to turn this into a robust backend for a larger app, the immediate improvements that have the highest payoff are:

1. **Expose geocoding alternatives:** allow `-limit > 1` and return candidates (or add an interactive/selection mode in a separate CLI).

2. **Persist last-request time in cache or state:** current orchestration resets `last_t` per run, so rate limiting is only meaningful inside a single process run.

3. **Add a validation/diagnostics mode:** e.g. include zenith RA/Dec, UTC offset, DST flag in JSON when `-debug` is passed.

4. **Add unit tests:** especially for DST edge cases (Europe/Rome and at least one non-European zone).

5. **Define a stable JSON schema:** even a simple `schema.json` + version field prevents pain later.

11 Appendix: example JSON (conceptual)

```
{
  "place": { "display_name": "...", "lat": 0.0, "lon": 0.0, "tzid": "Europe/Rome" },
  "time": { "local": "YYYY-MM-DDTHH:MM", "utc": "YYYY-MM-DDTHH:MMZ", "warnings": [] },
  "astro": {
    "zenith_constellation": "Monoceros",
    "sun": {
      "constellation": "Sagittarius",
      "ecliptic": { "lon_deg": 271.6, "lat_deg": -0.003 },
      "icrs": { "ra_deg": 309.6, "dec_deg": -19.44 }
    }
  }
}
```